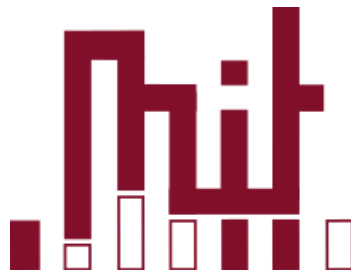


Beágyazott információs rendszerek (VIMIAD04)

Ütemezés, valós-idejű ütemezés

Budapesti Műszaki és Gazdaságtudományi Egyetem
Mesterséges Intelligencia és Rendszertervezés Tanszék



ZH információk

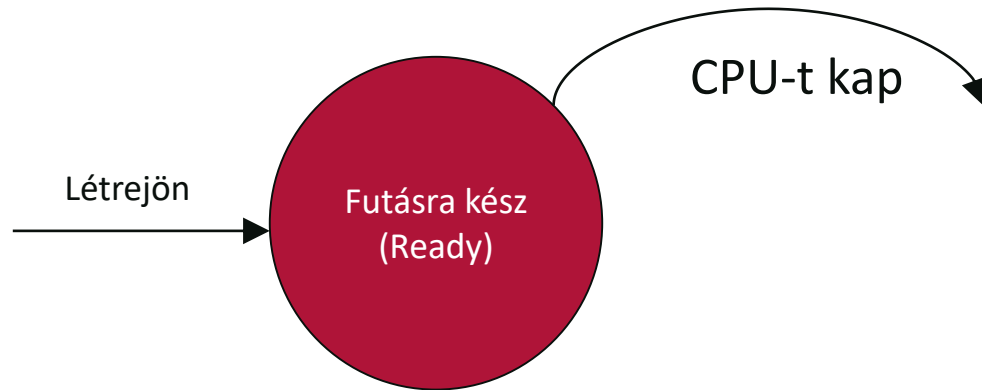
- ZH időpontja: 2026. 04. 28. (kedd), 8:15-10:00
 - Példa ZH a Teams-en
 - Kulcskonceptiók, stb. listája is fel fog kerülni
 - Két terem, névsor alapján besorolás terembe
 - Eredmény ASAP, ha minden jól megy, még aznap
 - Betekintés: Elektronikus (annotált ZH-t megkapjátok)
- Ütemezés:
 - ZH anyaga:
 - 6. főlíasorig (Beágyazott SW architektúrák)
 - 5. laborig (BeagleBone Black bemutató, GPIO használata)
 - Kérésre konzultáció a 2026. 04. 20-szal kezdődő hét második felében
 - Pl. 18:00-kor tudunk kezdeni csütörtökön
 - Ennek csak akkor van értelme, ha vannak kérdéseitek...

ÜTEMEZÉS

Hogyan/mikor fut a beágyazott SW?

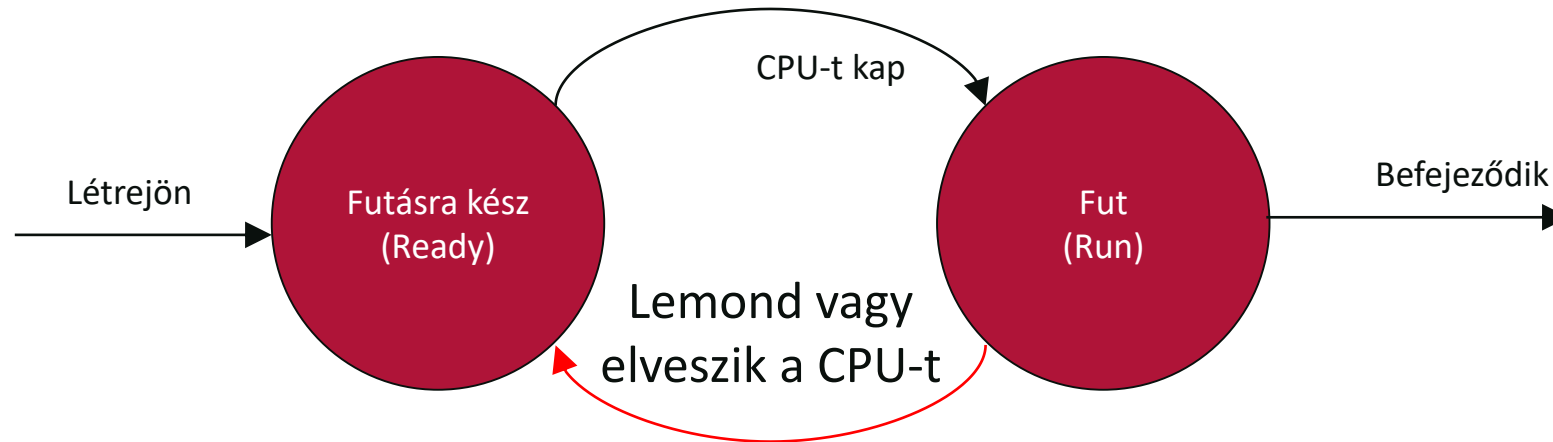
Ismétlés: Feladat állapota 3.

- Ha a CPU felszabadul, akkor egy futásra kész feladat futó állapotba kerülhet
- Milyen algoritmus dönti el, hogy a futásra kész feladatok közül melyik kap CPU-t? A CPU ütemezés (CPU scheduling)



Ismétlés: Feladat állapota 5b.

- A feladat lemond vagy elveszik tőle a CPU-t
- Lemond a CPU-ról. Pl. `yield()` rendszerhívás
- Elveszik a CPU-t. Preemptív CPU ütemezés, lesz róla szó...



Elveszik a CPU-t, miért, hogyan?

- A feladat fut, mi tudja elvenni a CPU-t?
- Az OS tudja csak elvenni → Futnia kell valahogy, de a folyamat fut...
- Megszakítás esetén tud futni az OS...
- **Kell egy megszakítás (HW, SW, vagy kivétel)**

Mi vezérli az állapotátmeneteket?

- Minden állapotátmenet megszakításra történik!
 - Megszakítás?
 - HW megszakítás, külső esemény, külső HW kér kiszolgálást
 - HW kiszolgálást kér egy fizikai jellel, pl. háttértárról adat beérkezett, hálózati csomag, keyboard-on egy billentyűt lenyomtak, stb.
 - Kivétel, belső esemény, a processzor állapotában változás miatt szükséges
 - Nullával osztás, túlcsordulás, laphiba, szegmentációs hiba, stb.
 - Szoftver megszakítás, belső esemény, rendszerhívás
 - Az SW kér kiszolgálást az OS-től
- A modern operációs rendszerek megszakítás vezéreltek!
 - Az OS mindent eseményként kezel, esemény-vezérelt!

VALÓS-IDEJŰ SZOFTVER

Valós-idejű szoftver aktivitás

- Nagyon leegyszerűsítve:
 - Egy olyan szoftver aktivitás (kódrészlet),
 - Ami egy adott külső esemény hatására,
 - Garantált válaszidővel teljesen lefut,
 - Remélhetőleg függetlenül a rendszerben jelen lévő többi szoftver aktivitástól, eseménytől (beleértve a külső eseményeket), és a rendszer állapotától.
 - Ezt szeretnénk elérni...
- Alapkövetelmény:
 - Az aktivitásra adható legyen a válaszidőnél rövidebb végrehajtási idő
 - A HW-tól (CPU, memória, stb.) biztos függ, de hogyan?
 - Sok algoritmust elve kizár ez a követelmény!
 - Run-to-complition („befejezésig fut” ?)
- Nem az egész SW valós-idejű, hanem jól meghatározott részei!
- A feladat ezeknek az aktivitásoknak megadott válaszidővel történő végrehajtása és a többi, nem valós-idejű aktivitásnak a hatékony végrehajtása (ár, fogyasztás, stb. korlátok is vannak)

Worst-Case Execution Time (WCET)

- A továbbiakban tételezzük fel, hogy az alkalmazásunkban:
 - A valós-idejű aktivitásokra megadható egy WCET érték, és azok az előírásoknak megfelelően futtathatók a rendszerben (ezt vizsgálni tudjuk algoritmikusan, mérésekkel, szimulációval vagy ezek kombinációjával)
 - A többi (nem valós-idejű) feladat végrehajtására van elég „maradék” erőforrása a rendszernek a valós-idejű feladatok végrehajtása közben/után
 - Azok sem éheződnek ki (starvation, később beszélünk róla)
 - Komoly matematikai analízissel is járhat az ütemezhetőség vizsgálata
 - Többnyire adott HW és (egyszerű) ütemezési algoritmus esetén válaszidők, stb. tekintetében el is lehet azokat végezni
 - Ebbe most nem megyünk bele, több előadás lenne
 - A gyakorlatban ez a megközelítés visszaszorulóban is van (komplex rendszerekben sok problémával kell szembenézni az alkalmazásuk során)
- De azért először nézzük meg, mi ennek a realitása?
 - Mármint hogy mit tudunk mondani egy kódrészlet végrehajtási idejéről?

Mi befolyásolja a végrehajtási időt?

- Klasszikus processzor modell:
 - Utasításfüggő, de adott utasításra állandó utasítás végrehajtási idő
 - Végrehajtás egységes késleltetéssel elérhető fizikai memóriában
 - Nincs megszakítás a rendszerben (polling van)
 - Egyprocesszoros rendszer
 - Nincs verseny a memóriáért, perifériákért
 - Nincs DMA (Direct Memory Access v. Data Movement Architecture)
 - Nincs verseny a memóriáért, perifériákért
 - A perifériák elérésének időzítése ismert
 - Stb.
- Teljesen determinisztikus működés
 - Felső korlát adható, ha az algoritmus nem iterál
 - Az egy szekvenciális utasítás sorozat
 - Vagy ha az iterációk számára adható felső korlát
 - De ekkor nagyon konzervatív lehet a megadott végrehajtási idő

Modern végrehajtási környezetek?

- ARM Cortex-A/X, x86/AMD64, stb.
- Nem determinisztikus a végrehajtóegység
 - Sokmagos végrehajtás (verseny az erőforrásokért, mint pl. memória, periféria)
 - Többszintű CACHE
 - Szuperskalár CPU
 - Utasításelágazás becslés (Branch prediction) és egyéb spekulatív végrehajtást gyorsító megoldás a CPU-ban (out of order execution, register renaming, stb.)
 - Virtuális tárkezelés, háttértár (HDD/SSD/stb.)
 - PCIE/USB buffering perifériák esetén
 - Erősen áteresztőképességre optimalizált architektúra (nem késleltetésre)
 - Sok buffer, stb.
- OS elkerülhetetlen, valós-idejű OS használatával biztosítható csak a valós-idejűség
 - SW végrehajtás bizonytalansága, megszakítják a feladatot (elveszik a CPU-t)
 - Ezt kezeljük majd a megfelelő ütemezési algoritmussal...

Modern végrehajtási környezetek?

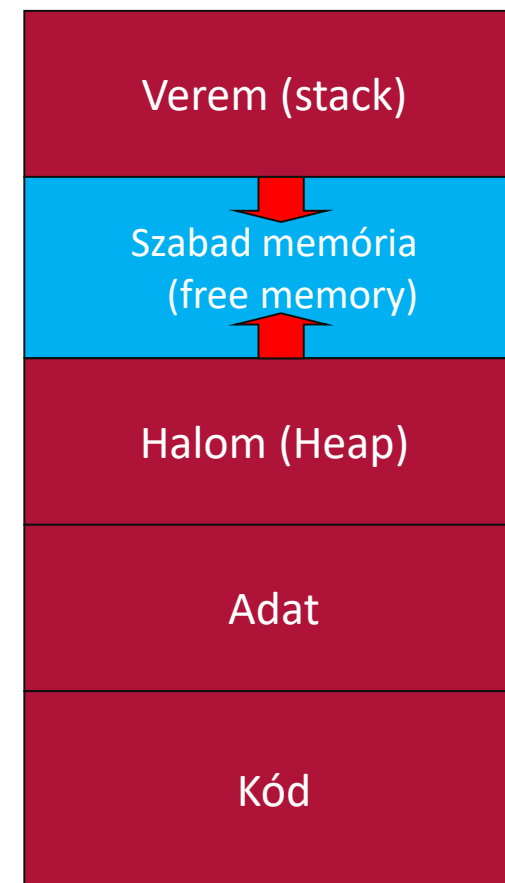
- Ha a virtuális tárkezelést kizárjuk, és az aktivitás nem tartalmaz háttértárkezelést, vagy ezeket erősen korlátozzuk, akkor többnyire adható felső korlát, de az erősen konzervatív!
 - A tipikus végrehajtási idő és a legrosszabb eset (worst-case) nagyságrendekkel eltér
 - A legrosszabb eset ritkán fordul elő, de „előfordul”
 - OS overhead is jelentős (a valós-idejű OS ebből a szempontból rosszabb, nagyobb az overhead-je)
 - Linux stress-ng hatása (tesztelésre, esetleg ezoterikusabb stressor-okkal)
- A megoldás: Heterogén architektúra...
 - Ami időkritikus, azt dedikált végrehajtóegységes hajtjuk végre
 - Megtehetjük, mert a modern félvezető technika mellett egy SoC-on egy extra, valós-idejű végrehajtó egység olcsón megvalósítható
 - Valós-idejű végrehajtó egység egyszerű: Cache nem kell, spekulatív végrehajtás nem kell, stb.
- Nem gondoltunk a hálózati kommunikációra, az alapértelmezetten nem valós-idejű!

FELADATOK MEGVALÓSÍTÁSA

Építőkövek...

Folyamat fogalma

- A folyamat a végrehajtás alatt álló program
 - Ugyanabból a programból több folyamat is létrehozható
 - Saját kód, adat, halom (heap) és verem memória területtel rendelkezik
 - Védettek a többi folyamattól
 - Szeparáció, virtuális gép, sandbox
 - Belépési pont:
 - `int main(int argc, char *argv[]) {}`



Folyamatok szeparációja

- Virtuális CPU-n futnak:
 - Nem férhetnek hozzá a többi folyamat és az operációs rendszer futása során előálló processzorállapothoz
 - Kontextus váltás történik, ha más folyamat kerül futásra
 - A CPU védelmi szintek és az OS együtt oldja ezt meg
- Saját virtuális memóriaterületük van
 - Nem férhetnek hozzá más folyamatok virtuális memóriájához vagy direkt módon a fizikai memóriához
 - A processzor MMU-ja és az OS együtt oldja ezt meg
 - Lehetséges azonos fizikai memóriaterületek megosztása például olvasási joggal (pl. kód terület)
 - A modern MMU-k ezt akár írási joggal is meg tudják tenni
 - Teljes MMU és OS kontroll alatt a biztonság érdekében

Folyamatok létrehozása

- OS specifikus rendszerhívás:
 - Windows: CreateProcess()
 - UNIX/Linux: fork() + exec() API család
 - FreeRTOS: Nem támogatja a folyamatokat!
- Szülő/gyermek viszony a létrehozó és a létrehozott között.
 - Process fa (process tree)
 - ps és pstree parancsok
 - A szülő erőforrásaihoz hozzáférés többnyire konfigurálható (mindenhez – semmihez)
 - A szülő megvárhatja a gyermek terminálódását, vagy futhat vele párhuzamosan
 - Paraméterezhető a gyermek (command line)
- Sok adminisztráció, erőforrás-igényes
 - /proc/... file-struktúra a Linuxban

Folyamatok kommunikációja

- A folyamatoknak együtt kell működniük
 - Ehhez kommunikálniuk kell
 - Inter-Process Communication (IPC)
- Két tetszőleges folyamat nem tud közös memórián keresztül kommunikálni
 - Az MMU és a virtuális memória éppen ezt kívánja lehetetlenné tenni (szeparáció)
 - Csak OS rendszerhívásokon keresztül tudnak kommunikálni, ami erőforrás igényes
- Hatékony a védelem/szeparáció szempontjából
- Nem hatékony módja a párhuzamos, erősen összefüggő feladatok megoldásának
 - Pl. akár GUI + számításigényes feladat (WORD „újrászedés”)

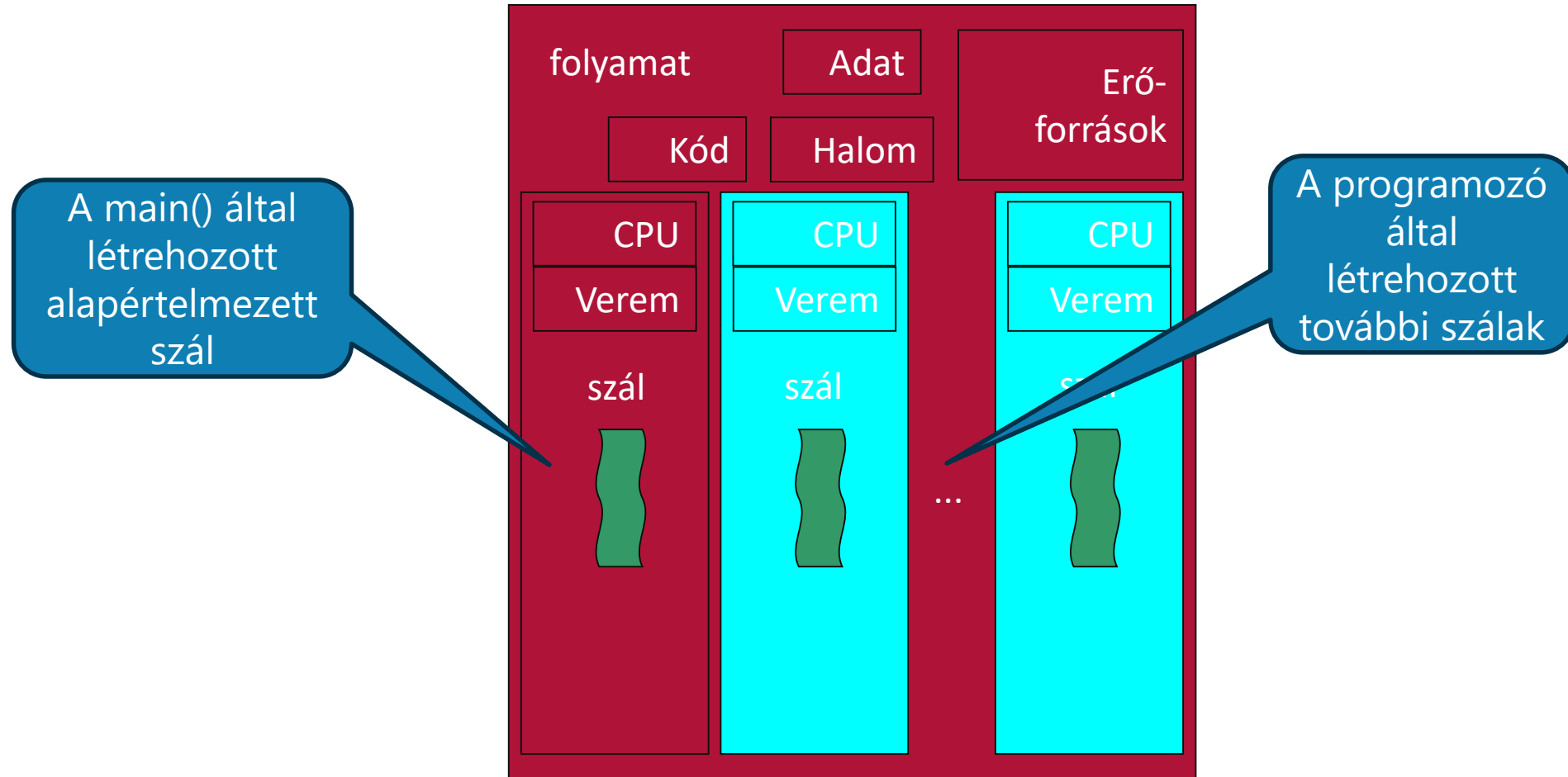
Folyamatok befejezése

- OS specifikus rendszerhívás:
 - Windows: TerminateProcess()
 - UNIX/Linux: exit(), return
- Nyitott, használatban lévő erőforrásokat le kell zárni
 - Pl. nyitott file-ok, stb.
- A szülő megkapja a visszatérési értéket, többnyire egy egész értékű változó (integer) formájában
- Mi történik, ha a szülő folyamat befejeződik, de a gyermek nem?
 - OS függő megvalósítás, tipikus megoldások:
 - Alapértelmezett szülő folyamat (pl. UNIX init folyamat)
 - A gyermek automatikus befejezése (cascading termination)
- Sok adminisztráció, erőforrás-igényes

Folyamatok értékelése, problémák megoldása

- Védelmi/szeparációs szempontból jó megoldás, de erőforrás-igényes:
 - Folyamat létrehozása és megszüntetése
 - Folyamatok közötti kommunikáció és erőforrás-megosztás
- Megoldás: Szál (thread) bevezetése:
 - A szál a CPU-használat alapértelmezett egysége, magában szekvenciális kód
 - Saját virtuális CPU-ja van, és saját verem áll rendelkezésre
 - A kód, adat, halom és egyéb erőforrások (pl. file) tekintetében osztozik azokkal a további szálakkal, amelyekkel **azonos folyamat kontextusában** fut
- Folyamat = nehézsúlyú folyamat (heavyweight process)
- Szál = pehelysúlyú folyamat (lightweight process)

Folyamatok és szálak



Többszálú rendszer folyamat alapon
(modern UNIX és Windows)

Szálak támogatása

- Jelenleg a modern operációs rendszerek natív módon támogatják szálak létrehozását
 - A folyamatban egyet az OS létrehoz a `main()` fv-nek
 - Utána szükség szerint a programozó létrehozhat továbbiakat
- Windows:
 - Program v. szolgáltatás = folyamat, folyamaton belül szálak
 - Az ütemező szálakat ütemez
- Linux:
 - Program v. daemon = folyamat, folyamaton belül szálak
 - daemon: rendszerfolyamat, nem egy user futtatja, hanem a rendszer „maga”, pl. WEB szerver, stb.
 - Az ütemező task-okat ütemez, amik lehetnek folyamatok vagy szálak
 - Kompatibilitás biztosítása a régi UNIX programokkal
 - Flexibilitás (később más ütemezhető entitások is létrehozhatók, kezelhetők)

Szálak támogatása (létrehozása, kezelése, megszüntetése)

- Pl. Win32 API, Pthreads, JAVA thread
- Win32: CreateThread() bonyolult paraméterezéssel + kezelő API
- Pthreads: POSIX threads pl. Linux és más UNIX variánsok kernel vagy akár user szinten (csak viselkedést ad meg)
 - A main() függvényben, programozottan
 - Lesz a laborban...

Szálak alkalmazásának előnyei

- Kis erőforrás igényű a létrehozásuk és megszüntetésük
 - Kb. egy nagyságrenddel gyorsabb mérések szerint
- Alkalmazáson belüli többszálúság támogatása
 - A GUI válaszol, még ha háttérben csinál is valamit az alkalmazás (pl. számol)
- Skálázhatóság
 - Alkalmazáson belül kihasználható több CPU
- Gyors kommunikáció közös memóriában az azonos folyamat kontextusában futó szálakra
 - Csak a verem szál specifikus, a többi osztott
 - A másik szál verméhez is hozzáférhet alapértelmezett esetben, amiből nagy problémák lehetnek

Szálak alkalmazásának következményei

- A közös memórián keresztüli kommunikáció veszélyes
 - A kommunikációra használt memóriaterületek (struktúrák vagy objektumok) konzisztenciája sérülhet
 - Kölcsönös kizárást meg kell oldani, amire az OS fog szolgáltatásokat nyújtani (Az API-ban, a rendszerhívásokra támaszkodva)
 - Erről már beszéltünk a múlt órán...
 - Eltérő folyamatok kontextusában futó szálak kommunikációja az OS-en keresztül történik
 - Erre ritkábban van szükség, mivel a szorosan összekapcsolódó (sokat kommunikáló) feladatok megvalósíthatóak egy folyamaton belül

VALÓS-IDEJŰ ÜTEMEZÉS

Hogyan/mikor fut a szoftver?

Feladatok ütemezése (task scheduling)

- Feladat: Kiválasztani a futásra kész feladatok közül a futót (visszatértünk a feladat megnevezéshez)
 - Statikus/dinamikus ütemezés
 - Dinamikus ütemezés a tárgy keretein belül
- A feladatokat a többnyire feladat sorokban (task queue vagy job queue) tároljuk
 - Ez a legegyszerűbb esetben tényleg egy FIFO, ami TCB típusú objektumokra/struktúrákra mutató pointereket tartalmaz
 - Pl. futásra kész sor, adott eseményre vár sor, stb.
 - Az ütemezés sorbaállási modellje (Queuing diagram)
 - Az ütemező algoritmus ezeken a struktúrákon dolgozva végzi el a feladatát (elsősorban a futásra kész soron)
 - Keresésről van szó, általános esetben NP teljes a probléma
 - Viszont kemény valós idejű a feladat még nem valós idejű operációs rendszerben is
 - Az overhead-et egy minimális szinten kell tartani!

Ütemezés időskálái 1.

- Rövid távú (short-term) v. CPU ütemezés
 - A futásra kész sorból választ egy futó állapotba átmenő feladatot
 - Minimum 10-20 ms-enként végrehajtásra kerül (időzítő megszakítás), de inkább gyakrabban
 - Részletesen beszélünk róla a Linux esetén...

Ütemezés időskálái, középtávú ütemező

■ Középtávú (medium-term) ütemezés

○ Swapping

- A rendszerben lévő feladatok memóriájának egyes részei kiírhatók háttértárra
- Több feladat fér el a fizikai memóriába (virtuálisan)
- Egy feladatra több fizikai memória juthat (amikor fut)

○ Mi van, ha a futó vagy futásra kész (ami várhatóan hamarosan futni fog) feladat egyes szükséges részei a háttértáron vannak?

- Nem tud futni, addig, amíg a szükséges memória részek vissza nem kerülnek a fizikai memóriába
- A középtávú ütemezés ezt irányítja...

○ Beágyazott rendszerekben

- Flash memóriát tönkre teheti (pl. eMMC, SD kártya, de egy SSD-nek ezt ki kell bírnia)
- Valós-idejűséggel összeegyeztethetetlen!
- Mit lehet ellene tenni?
 - Nem konfiguráljuk (nincs swap file vagy partíció, FLASH-en még jól tesz az élettartamnak is), egyébként is minden belefér a memóriába
 - Ki lehet zárni egy adott memóriaterület swap-elését

Ütemezés időskálái, hosszú távú ütemező

■ Crontab

- Feladatok indítása OS boot végén
- Feladatok indítása adott időnként (perc, óra, nap, stb.)
- Feladatok indítása eseményre

■ Systemd hosszú távú ütemező funkciói

- Pl. folyamatok megfigyelése, leállás esetén újraindítás, stb.
- Nagyon összetett, rengeteg funkcióval rendelkező rendszer, nincs rá idő, hogy a részletekbe belemenjünk...

Ütemező választás

- Nincs univerzális ütemező algoritmus
- A feladatnak megfelelő ütemezőt kell kiválasztani
- A legtöbb beágyazott operációs rendszer erre lehetőséget ad
 - Az ütemező kiválasztható és testre szabható az operációs rendszer konfigurálása (fordítás előtt) során
 - Vagy futási időben, a már bekonfigurált ütemezők közül tudunk választani

Linux ütemező (3.x verziótól) 1.

- Kernel verzió és konfigurációs opció függő
 - Részben fordítási, részben kernel boot paraméterrel is konfigurálható
- Egy és több processzoros homogén architektúrájú rendszerekre optimalizált
 - Azonos ISA-t feltételez a magokra, órajelben, kisebb architektúrális részletben stb. lehet eltérés
 - Pl. ARM-Ax multicore Big-Little architektúrával, modern Intel/AMD P/E core-ok
- Ütemezési osztályok (osztály prioritás növekvő sorrendjében felsorolva)
 - Nagyon leegyszerűsítve
 - SCHED_OTHER:
 - Minden feladat, ami nem speciálisan lett egy más osztályba felsorolva
 - 2.6.23-tól CFS szerint ütemezve (2007, azóta több lépésben optimalizált)
 - 6.6-tól EEVDF
 - SCHED_RR: Valós-idejű RR ütemező
 - Szinte mindig jelen van a kernelben
 - SCHED_FIFO: Valós-idejű FIFO ütemező
 - Szinte mindig jelen van a kernelben
 - SCHED_DEADLINE: Valós-idejű Earliest deadline first ütemező (3.14 kerneltől, 4.16 update, 6.12 újabb update)
 - Tipikusan nincs befordítva a kernelbe
 - Fejlesztés alatt, pl. javított SCHED_DEADLINE a 6.12-es kerenelektől

CFS és EEVDF

- 2.6.23-tól az u.n. Completely Fair Scheduler (weighted fair queuing)
 - Bináris fában (red-black tree) tárolt sched_entity struktúrák
 - Folyamatosan fejlesztik, javítják, optimalizálják
 - Ütemezési csoportok (scheduling groups)
- Earliest Eligible Virtual Deadline First" (EEVDF) ütemező
 - 6.6-tól jelenik meg
 - A 6.12-től lett funkcionálisan teljes
 - A 6.14-es az aktuális
 - Az átlagos várakozási idő csökkentése a cél a késleltetés kritikus feladatoknál

Preempt-RT

- Preempt-RT Patch volt eredetileg
 - Hard real-time kernel, meg kell patch-elni, majd konfigurálni, majd lefordítani
 - Hol szakítható meg a kernel?
 - Áteresztőképesség szempontjából: Minél kevesebb helyen, mert akkor a megszakítás overhead alacsony szinten tartható
 - Késleltetés szempontjából: Minél több helyen, mert akkor a rendszer kis késleltetéssel tud reagálni a megszakításra, az overhead nem érdekes
 - Eltérő optimalizációs cél: Áteresztőképesség v. késleltetés
- Bekerült a 6.12-es kernelbe, azóta nem kell kernel patch
 - Kérdés a konfiguráció
 - Mi konfigurálható kernel parancssorból, milyen konfigurációhoz kell a kernelt újrafordítani

Többprocesszoros ütemezés

- Multiple-processor scheduling
- Homogén többprocesszoros rendszerekkel foglalkozunk.
 - Azonos alap utasításkészlet architektúra (ISA)
 - Utasítás készlet kiterjesztésekben (pl. AVX típus) lehet eltérés
 - Lehet SMP vagy NUMA architektúra a memória szempontjából (vagy azok keveréke).
 - Többnyire egy adott I/O periféria egy adott processzorhoz van rendelve (arra csatlakozik)
 - De lehet többhöz is (pl. nagysebességű hálózati kártyák)
 - Két lehetséges megoldás:
 - Master and slaves (egy CPU osztja ki a feladatokat)
 - Self-scheduling / peering (minden CPU ütemez)
- Az ütemezés heterogén többprocesszoros rendszerekben napjaink egyik legnagyobb szakmai kihívás
 - Sokféle ilyen rendszer van, nincs univerzális megoldás
 - Nehezíti a helyzetet a modern CPU-k big.LITTLE magokból álló felépítése
 - Teljesítmény/fogyasztás optimalizáció bonyolult, a szempontok ellentétesek
 - Eltérő lehet a támogatott ISA kiterjesztés készlet

Processzor affinitás (Processor Affinity)

- A CPU magok vagy mag csoportok rendelkeznek CPU mag vagy mag csoport specifikus CACHE-ekkel
 - A CACHE sikeres használata alapvetően meghatározza a CPU sebességét
- A cache a futó feladat utasítás és adat tartalmának egy részét tartalmazza
 - A processzor magok vagy mag csoportok cache tartalma különböző lehet (architektúrától és a rajta futó feladatoktól függ)
 - A cache koherencia csak a közösen tartalmazott adatokra vonatkozik, a teljes azonosságnak nem lenne értelme
 - A feladat más processzorra, vagy processzor magra kerülése csökkenti a végrehajtás sebességét (a lokális cache-ben nincs benne a kód és/vagy adat)
 - Az új CPU cache tartalmat ismét fel kell építeni...
 - Cél: A feladatot ugyan azon a végrehajtó egységen tartani (sebesség, késleltetés)
 - De cél a minél kisebb számú, minél alacsonyabb órajelű CPU használata is (fogyasztás)
- Laza vagy kemény processzor affinitás (soft or hard processor affinity)
 - Laza: Nincs garancia, de törekszik rá az OS (többnyire alapeset)
 - Kemény: Biztosan ugyan azon a CPU-n marad (rendszerhívással)

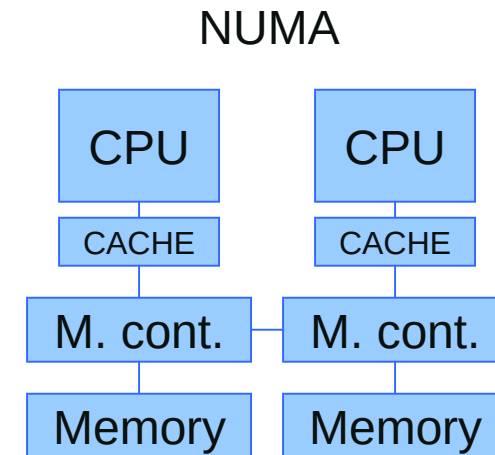
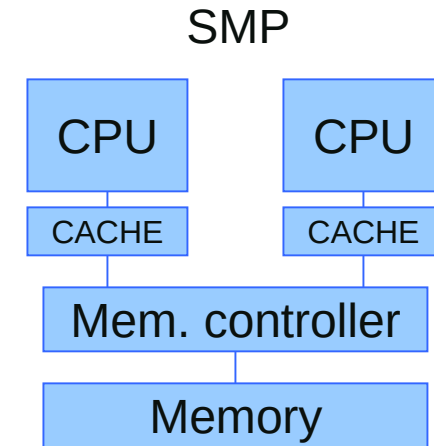
Processzor affinitás (Processor Affinity)

- A CPU magok vagy mag csoportok rendelkeznek CPU mag vagy mag csoport specifikus CACHE-ekkel
 - A CACHE sikeres használata alapvetően meghatározza a végrehajtás sebességét
- A cache a futó feladat utasítás és adat tartalmának egy helyi tárolója
 - A processzor magok vagy mag csoportok cache tárolója (architektúrától és a rajta futó feladatoktól függ)
 - A cache koherencia csak a közösen tartalmazott adatok között létezik, a teljes processzorra nem vonatkozik, a teljes rendszerre nem lenne értelme
 - A feladat más processzorra, vagy processzor magra való átváltása csökkenti a végrehajtás sebességét (a lokális cache-ben nincs benne a kód és/vagy adat)
 - Az új CPU cache tartalmat ismét fel kell építeni...
 - Cél: A feladatot ugyan azon a végrehajtó egységen tartani (sebesség, késleltetés)
 - De cél a minél kisebb számú, minél alacsonyabb órajelű CPU használata is (fogyasztás)
- Laza vagy kemény processzor affinitás (soft or hard processor affinity)
 - Laza: Nincs garancia, de törekszik rá az OS (többnyire alapeset)
 - Kemény: Biztosan ugyan azon a CPU-n marad (rendszerhívással)

Demó Task Managerrel
Windows-on (láthatóbb):
Intel Core i5 1135G7
(4 mag + HT)

SMP v. NUMA

- SMP esetén a processzor affinitás csak a cache találatok szempontjából érdekes
- NUMA (processzor csoport) esetén a feladat által használt fizikai memória is érdekes a processzor affinitás szempontjából
 - A feladat elsősorban a CPU-hoz közeli memóriából fusson
 - A távoli (más CPU-hoz csatlakozó) memória használata minimalizálандó
 - Összetettebb ütemezési algoritmust igényel



Terhelés megosztás (load balancing)

- Egy globális futásra kész sor vagy processzoronkénti futásra kész sor?
- Processzoronkénti futásra kész sor a modern rendszerekben
 - Push and/or Pull
 - Push: OS kernel folyamat mozgatja a sorok között a feladatokat.
 - Pull: Az idle állapotban (idle feladatot végrehajtó) CPU próbál a többi sorából feladatot kapni.
 - A kettő kombinációja is használható.
- Linux: processzoronként sorok, affinitás programozottan beállítható, push/pull terheléselosztás
- Összefüggő, párhuzamosan futtatható feladatok optimalizálása
- ARM/AMD/Intel big.LITTLE CPU-k esetén képességek figyelembe vétele
 - Teljesítmény-fogyasztás optimalizálás
 - Eltérő utasítás készlet architektúra kiterjesztés támogatása
 - Pl. bizonyos AVX kiterjesztések nem érhetők el a hatékonyságra (kis fogyasztásra optimalizált magokon), ide azt használó kód nem ütemezhető
 - Erősen függ a jó ütemezőtől, ami Linux esetén kernel függő (minél újabb, annál jobb, többnyire)

Ütemező befolyásolása

- Shell parancsok
 - A `nice` program
 - A `chrt` program
 - Egy folyamat valós-idejűséggel kapcsolatos tulajdonságainak a befolyásolására
 - SCHED_RR/SCHED_FIFO/SCHED_DEADLINE és azok paraméterei állíthatók
 - Nem kell hozzá a program valós-idejű megírása, de ha nem az, baj lehet...
- A root jogosultságú program programozottan rendszerhívásokkal szál szinten tudja befolyásolni az ütemezőt
 - Pl. egy szál áthelyezhető/indítható SCHED_RR, SCHED_FIFO, SCHED_DEADLINE ütemezési osztályba/osztályban
 - Nem megyünk bele a részletekbe, akinek kell, majd megnézi
- Többprocesszoros rendszer
 - Processzor affinitás (rendszerhívás: `sched_setaffinity`, `taskset cli app`)
 - CPU mag kivonható a standard ütemezésből (kernel command line), majd adott program explicit futtatható az adott CPU-n

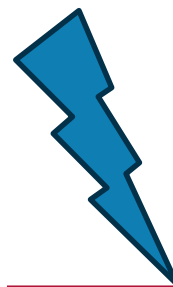
Az ütemező és az idő

- Számos ütemező algoritmusnak követnie kell az idő múlását
 - Round-Robin, Deadline (STR, LST)
 - Időmérés, időtúllépés (timeout) meghatározása
 - Részfeladatok periodikus vagy adott időpontban történő futtatása
- Az ütemező ezekben az esetekben egy periodikus HW Timer megszakítást használ (system ticks, Linux : jiffies)
 - Periodikusan tud futni, és a szükséges funkciókat meg tudja valósítani
 - Gyakoriság:
 - Minél kisebb a periódusidő, Annál nagyobb felbontású ütemezés/időmérés lehetséges, de annál nagyobb az overhead (annál gyakrabban fut)
 - A default 4 ms, de szükség esetén állítható, egyes disztribúciókban pl. 1 ms
- Timer szolgáltatás
 - Virtuális timer a részfeladatok számára
 - Időpontra, adott időintervallum elteltére lehet várni (eseményként)
 - Az ütemező HW Timer-jéből vagy más timer-ből származtatva
 - A Linux-ban több/sok fajta van
 - És sok rendszerhívásnak lehet timeout-ot is megadni
 - Amikor egy idő túllépés hibával térnek többnyire vissza

Virtuális/SW Timer

- Sleep()/usleep/nanosleep(), poll() timeout, stb.
- Egy rendszerben rengeteg ilyen kérés érkezik az OS-hez párhuzamosan
- Hogyan lehet ezeket hatékonyan implementálni?
 - Várakozási kérések tárolása futási időponttal
 - $\text{Futási időpont} = \text{Kérés időpontja} + \text{Kért várakozási idő}$
 - Várakozó taskok listája futási időpont szerinti sorba rendezve,
 - Az elejét kell megnézni, ha a megadott idő után vagyok, a task mehet futásra kész állapotba
- Felbontás, pontosság?
 - Megszakítás gyakoriság adja meg a felbontást
 - Kvantálási hiba
 - Két system tick IT között érkezik a kérés
 - Csak egész Systick-et tudunk várni
 - Csak futásra kész állapotba kerül, utána az ütemező függ, mikor fut

Minden system tick IT-re ellenőrizzük



Futási időpont (Kérés időpontja + kért várakozási idő)	Várakozó task azonosítója (TaskID)
Legkorábbi időpont	Kérő TaskID
Legkésőbbi időpont	Kérő TaskID

Időmérés...

- System tick IT-ben system time karbantartása ($t_{\text{new}} = t_{\text{old}} + T_{\text{systick}}$)
 - CLOCK_MONOTONIC_RAW, CLOCK_MONOTONIC, CLOCK_REALTIME, és egyéb „órák”
 - Valós felbontás: system tick
- Hogyan tudunk ns felbontással időt lekérdezni?
 - System tick-et egy HW timer (számláló) állítja elő valamilyen órajelből, annak a HW számlálója lekérdezhető (perifériaként)
 - Az alapján „számítható”, hol tartunk két system tick között
 - A felbontás az órajel periódusideje T_{clk} , ami tipikusan 100-10 ns között van (10-100 MHz)
 - Timestamp Counter a CPU-ban (64 bites)
 - A CPU névleges órajelével jár, viszont független a többi órától
 - » Bekapcsolás óta eltelt órajel impulzusok számát adja meg)
- Tickless kernel: Az óraütes kezelésével járó fogyasztás jelentős lehet alacsony átlagos terhelésnél (pl. okostelefon)
 - Nincs periodikus óraütes, ha nincs CPU intenzív taszk
 - Külső esemény ébreszt (hálózati csomag, UI esemény, adott időpontra beállított HW timer, stb.)

HW System Clock

- Intel Always Running Timer (ART)
 - X86_FEATURE_ART
 - HW System Clock (CLOCK_REALTIME is)
 - Lényegében egy speciális regiszter
 - Támogatás
 - A 7. generációs Intel CPU-kon jelenik meg
 - A 11. generációtól van rendszerszintű integráció (PCIe/Ethernet HW sync)
 - Külső szinkronizációs PIN-ek (periféria szinkronizáció)
 - Time-aware General-Purpose Input/Output (TGPIO)
- Később, az idővezérelt működősnél kerül elő...

ÜTEMEZÉS SORÁN FELMERÜLŐ PROBLÉMÁK

Párhuzamos programozásból következő hibák

- Versenyhelyzet (race condition):
 - A párhuzamos program lefutása során a közös erőforrás helytelen használata miatt a közös erőforrás vagy az azt használó program nem megfelelő (hibás) állapotba kerül
 - Ilyen volt a korábbi összetett adattípusos példa
 - A hibás állapot részletei erősen függenek az azt használó szekvenciális részfeladatok lefutási sorrendjétől
- Minden párhuzamos programozási hiba versenyhelyzet
 - A hiba nehezen, statisztikailag reprodukálható csak, az időzítési és konkurencia viszonyoktól függően
 - Hol jó, hol nem jó. Plusz Murphy...
 - Teljesen eltér a klasszikus programozási hibáktól
 - Azonos bemeneti paraméterekre mindig azonosan viselkedik

Kiéheztetés (starvation)

- Ha a párhuzamos rendszer hibás működése miatt egy feladat soha nem jut hozzá a működéséhez szükséges erőforrásokhoz, akkor ki van éheztetve (nem tud futni)
- Nem csak a CPU-ra, de más közös erőforrásokra is felmerülhet
 - CPU-ra "Futásra kész" állapotban vár
 - Az összes többi erőforrásra „Eseményre várakozik” állapotban vár
 - A CPU-ra és eseményre várakozó feladatok vizsgálatával lehet a helyzetet felismerni...

Holtpont (deadlock) és livelock

■ Holtpont:

- A közös erőforrások hibás beállítása vagy használata miatt a rendszerben a részfeladatok egymásra várnak
- Nincs futásra kész folyamat
- Nem jöhet létre belső esemény
- A rendszer nem tud előrelépni
- Példa: „dining philosophers”

■ Livelock (nincs jó magyar neve):

- Többnyire a hibás holtpont feloldás eredménye
- Kölcsönös kizárás + timeout ide vezet
- A rendszer folyamatosan dolgozik, de nem lép előre
- Példa: „Két kedves ember összetalálkozik az ajtóban vagy két kiskecske a hídon”

Holtpont (deadlock) beágyazott rendszerekben

- A holtpont a legsúlyosabb hiba, ami párhuzamos programozás során előfordulhat
- Pontos definíció:
 - Egy rendszer feladatainak egy H részhalmaza holtponton van, ha a H halmazba tartozó valamennyi feladat olyan eseményre vár, amelyet csak egy másik, H halmazbeli feladat tudna előállítani
 - H részhalmazról van szó, többnyire csak néhány feladat érint
- Beágyazott rendszerekben katasztrofális eredménnyel járhat
 - El kell kerülni, a rendszer lefagyásával jár!
 - A holtpontmentes működést a biztonságkritikus rendszerek fejlesztése során bizonyítani is kell...

Tipikus példa

- 2 erőforrás, A és B
- 2 feladat akarja őket használni az alábbi módon

P1 folyamat:

Lefoglal(A) ;

Lefoglal(B) ;

use A and B;

Felszabadít(A) ;

Felszabadít(B) ;

P2 folyamat:

Lefoglal(B) ;

Lefoglal(A) ;

use A and B;

Felszabadít(B) ;

Felszabadít(A) ;

Tipikus példa

- 2 erőforrás, A és B
- 2 feladat akarja őket használni egymás után

Versenyhelyzet (race condition) P1 és P2 között.

P1 folyamat:

Lefoglal(A) ;

Lefoglal(B) ;

use A and B;

Felszabadít(A) ;

Felszabadít(B) ;

P2 folyamat:

Lefoglal(B) ;

Lefoglal(A) ;

use A and B;

Felszabadít(B) ;

Felszabadít(A) ;



Tipikus példa

- 2 erőforrás, A és B
- 2 feladat akarja őket használni egymással szembe fordított sorrendben

Versenyhelyzet (race condition) P1 és P2 között.

P1 folyamat:

Lefoglal (A) ;

Lefoglal (B) ;

use A and B;

Felszabadít (A) ;

Felszabadít (B) ;

P2 folyamat:

Lefoglal (B) ;

Lefoglal (A) ;

use A and B;

Felszabadít (B) ;

Melyik fut előbb?

Tipikus

Azonos sorrendbe lefoglalva az erőforrásokat, a hiba nem jelentkezik!

De többnyire az erőforrások neve nem A és B, nem ilyen egyértelmű az összefüggés...

- 2 erőforrás
- 2 feladat akarja őket használni az

P1 folyamat:

Lefoglal(A) ;

Lefoglal(B) ;

use A and B;

Felszabadít(A) ;

Felszabadít(B) ;

P2 folyamat:

Lefoglal(A) ;

Lefoglal(B) ;

use A and B;

Felszabadít(B) ;

Felszabadít(A) ;

Holtpont a gyakorlatban

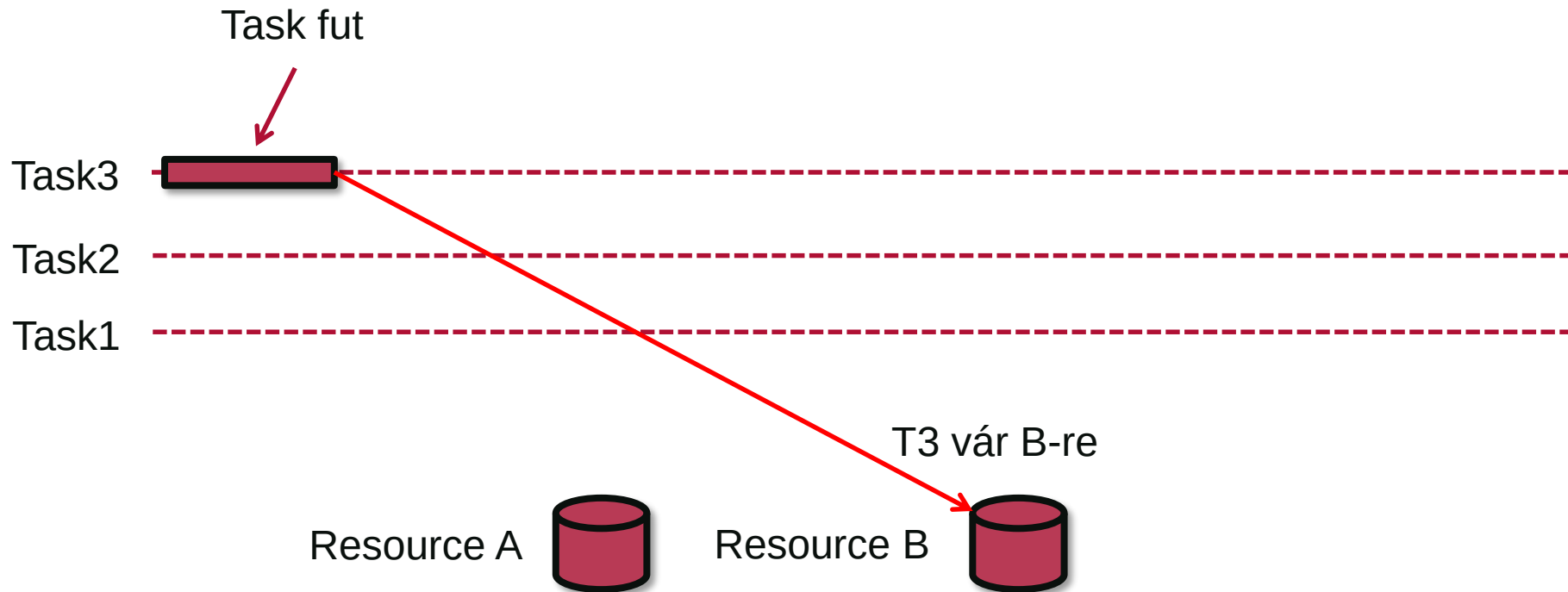
- Az egyszerű példánál sokkal összetettebb esetek is előfordulhatnak
- Nehéz felismeri...
 - Versenyhelyzet formájában jelentkezik
 - Hol működik, hol előáll a holtpont...
 - Bizonyos feltételek együttes teljesülése esetén áll elő a holtpont
- A holtpont más feladatokat is befolyásolhat:
 - A H részfeladatok által lefoglalt és soha fel nem szabadított (befagyott) erőforrásokon keresztül

Prioritás inverzió

- Prioritás inverzió (priority inversion):
 - Prioritásos ütemezőkben fordulhat elő, de az erőforrás használattal is összefügg.
 - A legegyszerűbb esetének előfordulásához kell:
 - 3 feladat, különböző statikus prioritással,
 - Egy közös erőforrás, amelyet a 3 feladat közül a legmagasabb és a legalacsonyabb is használni kíván.
 - A közepes prioritású feladatnak CPU intenzívnek kell lennie.
 - Kedvezőbb esetben csak a rendszer teljesítménye csökkent, a válaszidők nőnek.
 - Valós idejű rendszer????
 - Rosszabb esetben kiéheztetés, vagy akár holtpont is lehet a prioritás inverzió eredménye.
 - Klasszikus példa: Mars Pathfinder 1997...

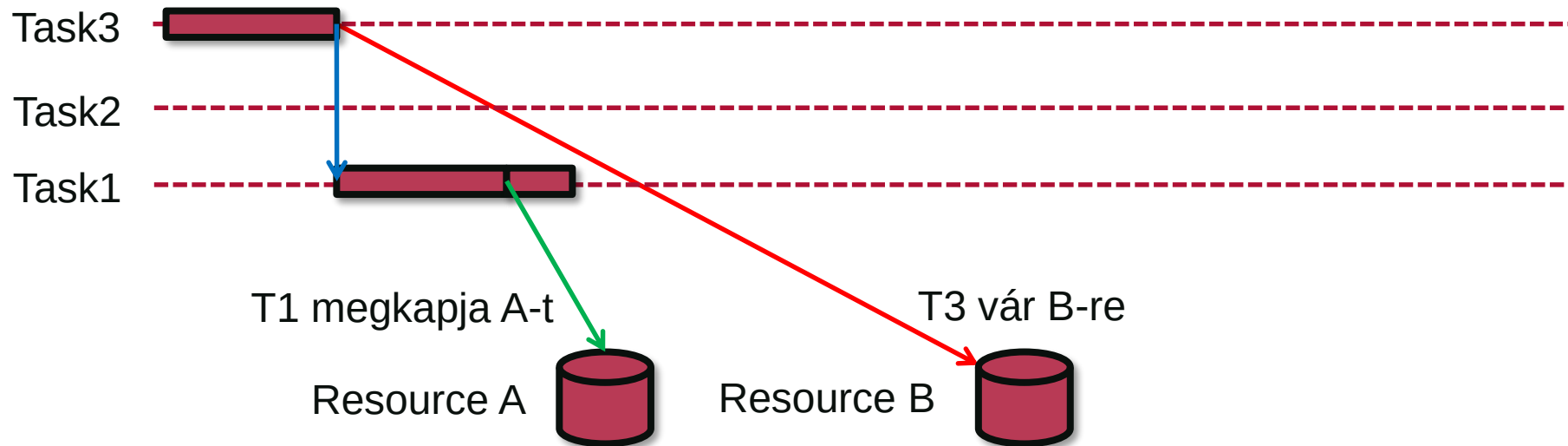
Prioritás inverzió példa 1.

- Lépések sorozata:
 - Task3 magas prioritású feladat valamilyen eseményre vár (de nem A erőforrás felszabadulására). Pl. B erőforrásra



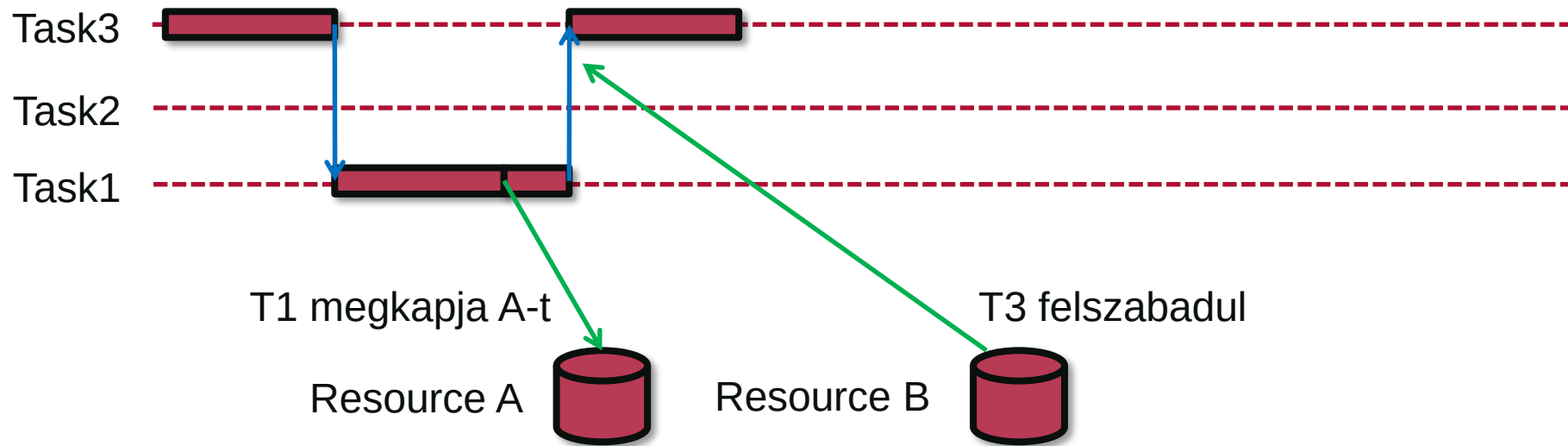
Prioritás inverzió példa 2.

- Lépések sorozata:
 - Task1 alacsony prioritású feladat fut, és megszerzi az A erőforrást, és azt használva fut tovább



Prioritás inverzió példa 3.

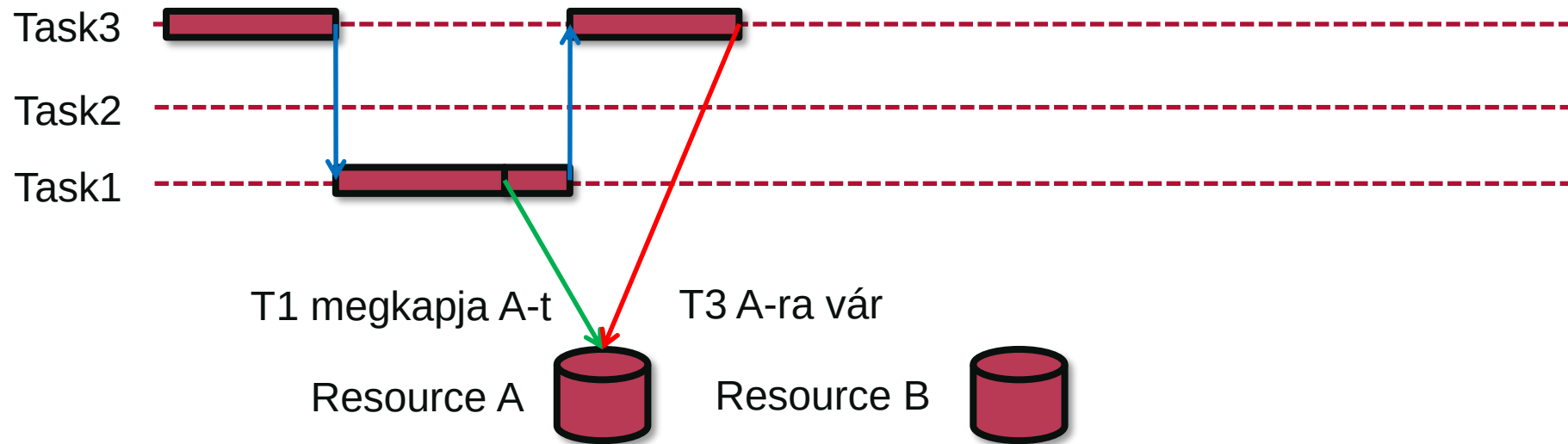
- Lépések sorozata:
 - Task3 magas prioritású feladat által várt esemény megérkezik (B felszabadul), futni kezd (preemptív ütemezés)



Prioritás inverzió példa 4.

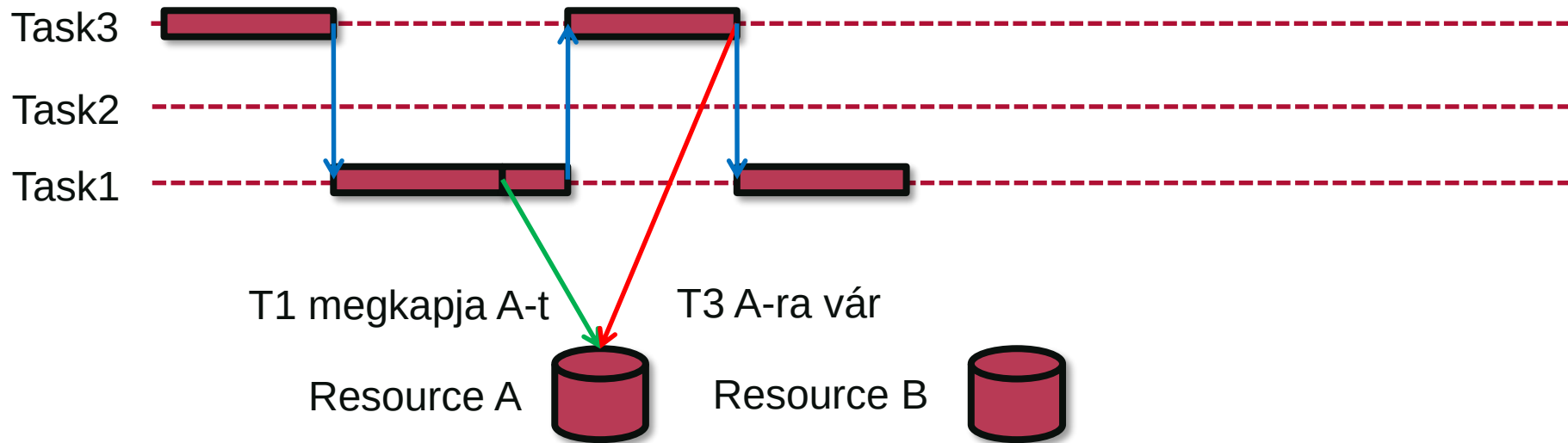
- Lépések sorozata:

- Task3 fut, majd használni kívánja az A erőforrást. Mivel A foglalt, ezért várakozni kezd (lényegében T1-re vár)



Prioritás inverzió példa 5.

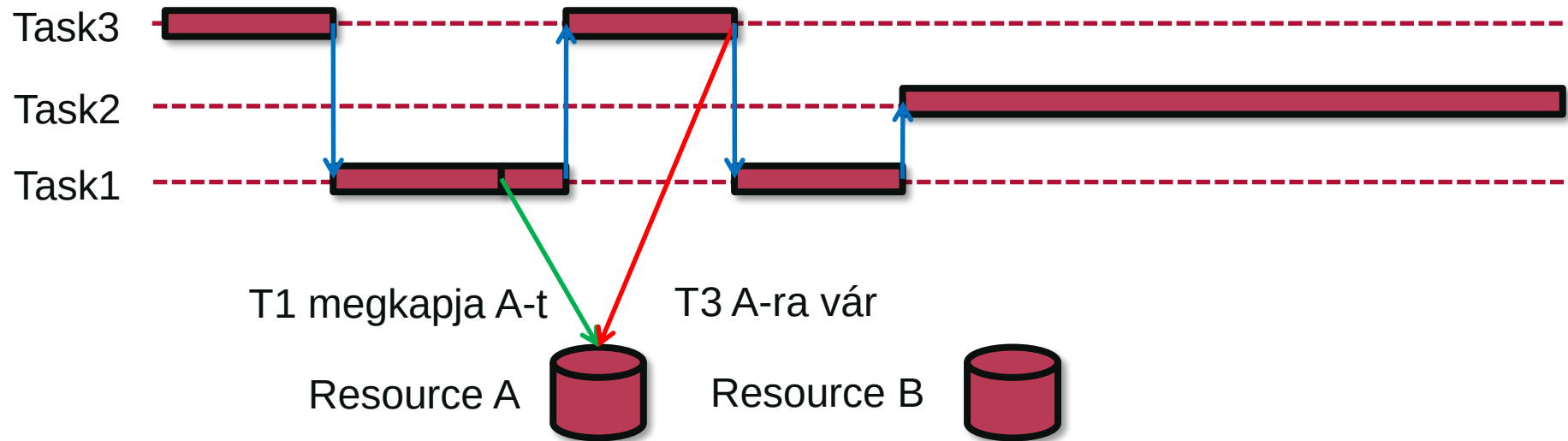
- Lépések sorozata:
 - Task1 ismét tud futni (nincs magasabb prioritású feladat), használja A-t.



Prioritás inverzió példa 6.

- Lépések sorozata:

- Task2 közepes prioritású feladat futásra kész állapotba kerül, és mivel magasabb prioritású, futó állapotba kerül, és hosszú (akár végtelen) ideig CPU intenzív feladatokat hajt végre (I/O burst nélkül).

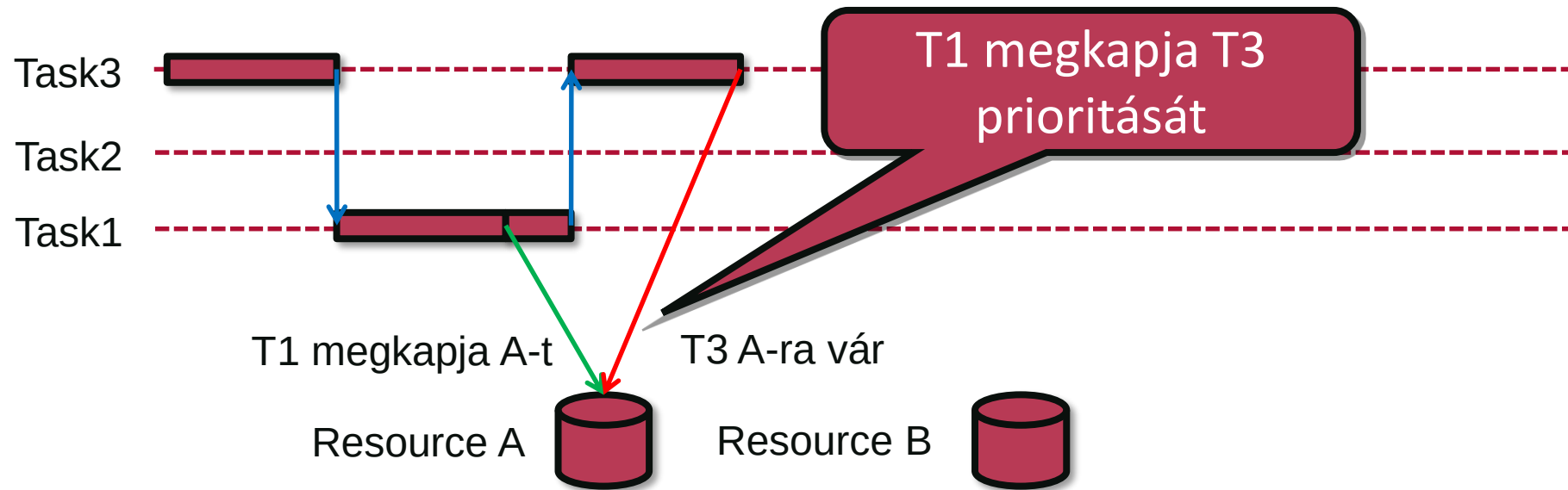


Prioritás inverzió végeredmény

- **Eredmény:**
 - Task3 nem tud továbblépni, hiszen eseményre vár (A erőforrás felszabadulására).
 - Task1 nem tud továbblépni, hiszen CPU-ra vár.
 - Task2 (TaskX) nem foglalkozik az A erőforrással, és amíg intenzíven használja/használják a CPU-t:
 - Task1 nem tudja befejezni a munkáját és felszabadítani az A erőforrást.
 - Amíg A erőforrás nem szabadul fel, Task3 nem tud futni, és végrehajtani a magas prioritású feladatát.
- **Task3 magas prioritású feladatot alacsonyabb prioritásúak nem hagynak futni... ☹️, 🙅, 💣, ☠️**
- **Ez egy egyszerű eset, ennél összetettebb módon is előállhat ez a helyzet!**

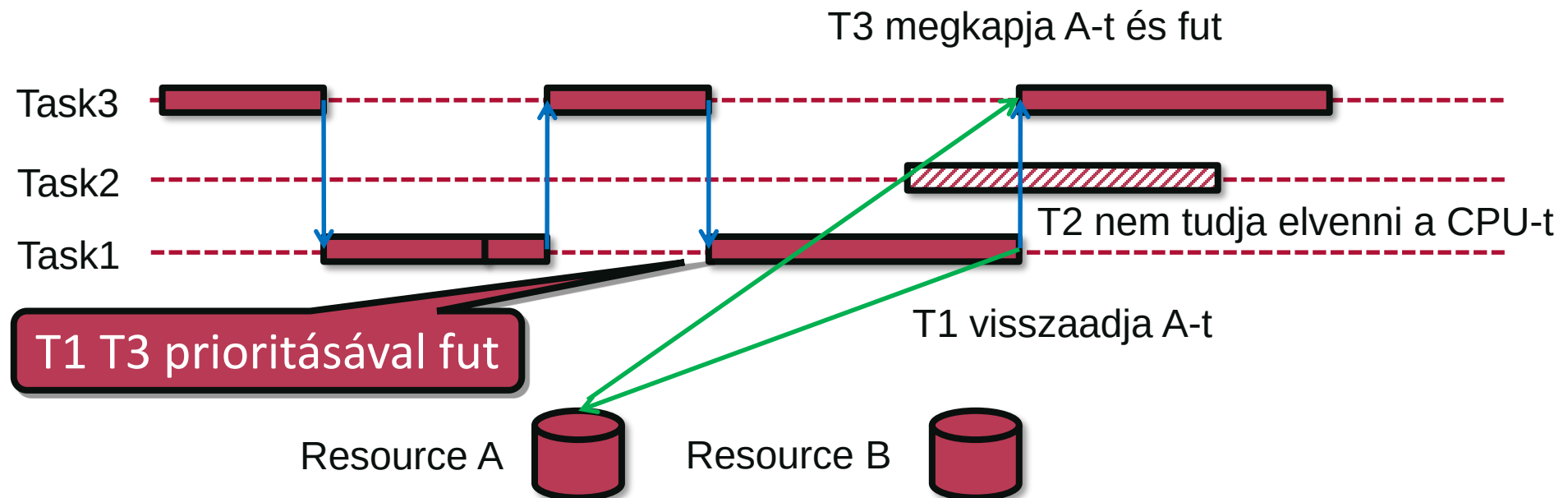
Megoldás 1: Prioritás öröklés (priority inheritance, PI)

- Prioritás inverzió példa 4. fázistól nézzük...
- Eredeti:
 - Task3 fut, majd használni kívánja az A erőforrást. Mivel A foglalt, ezért várakozni kezd (lényegében T1-re vár)
- Prioritás öröklés: T1 megörököli T3 prioritását, amikor az A-ra kezd T3 várni...



Prioritás öröklés hatása

- T1 futni kezd, de amikor T2 futásra készvé válik, T1 T3-tól örökölt magasabb prioritása miatt nem tudja elvenni a CPU-t
- T1 lefut, T3 megkapja a CPU-t



Megoldás 2: Prioritás plafon (priority ceiling, PC)

- Prioritás plafon (Priority ceiling, PC):
 - Majdnem ugyan az, de az adott közös erőforrást használó legnagyobb prioritású feladat prioritását örökli meg (ami lehet nagyobb mint az éppen feltartott feladat).
 - Ezt meg kell határozni, a fordítás során ez egy „paraméter” az erőforrásra vonatkozóan
 - Ilyen értelemben a PC algoritmus „fordítási idejű” (a PI futási idejű)
 - Az adott erőforrást máskor használó többi feladat sem tud futni (ha esetleg azok is CPU intenzívvé „válnak”).
 - Az alacsony prioritású feladat akadályoztatás nélkül le tud futni.
- Modern beágyazott OS-ekben választhatók...
 - None, PI, PC (ha preemptív ütemezőt választunk)

Megoldás

- Így javították meg a Mars Pathfinder-t is. Lényegében a prioritás öröklést beállították, és újrafordították az OS-t, aztán letöltötték a patch-et (diff) MARS-on lévő HW-ba...
- Miért nem használták egyből? : Nem tudták hogy van ilyen jelenség, a VxWorks-ben meg nem ez volt a default (overhead-je nagyobb).
Miért nem fedezték fel tesztelés közben? : Mert soha nem teszteltek valóshoz hasonló terhelési körülmények között. A teljes adatgyűjtés soha nem ment a földön, csak részrendszerek (gratulálok...).
- ...
- Modern beágyazott OS-ekben
 - None, PI, PC (ha preemptív ütemezést választunk)

kizárással feltartott

gnagyobb prioritású
pen feltartott

utni (ha esetleg azok

a futni.